# Arbitrary Precision Arithmetic Engine

Budde Dhathvi Charan
CS24BTECH11016
Department of Computer Science
IIT Hyderabad

May 1, 2025

# Introduction

In most modern programming languages, dealing with numbers is simple — until it's not. When the numbers involved are exceptionally large, or require a high degree of decimal precision, built-in types like 'int' or 'float' start to fall short. Whether you're working in cryptography, scientific modeling, or financial calculations, there are times when **"close enough" isn't good enough**.

This goal of this project is simple: to go beyond the default limitations of number handling in Java. The idea is to build a system that could perform arithmetic operations — addition, subtraction, multiplication, and division — on numbers of arbitrary size and precision. No shortcuts, no built-in BigInteger or BigDecimal. Just strings, logic, and a lot of manual computation.

Through this project, not only did I learn how to implement arithmetic operations from scratch, but I also gained a deeper understanding of OOP. This report documents the design, implementation, and results of that system.

# Abstract

This project presents a Java-based arbitrary-precision arithmetic engine capable of performing addition, subtraction, multiplication, and division on very large integers and high-precision decimal numbers. The implementation involves string-based manipulation, custom logic for digit-wise operations, and careful handling of negative values and decimal scales. It also includes a Python script and build automation with Ant for simplified usage. This report explains the design decisions, implementation logic, and results of test cases.

# 1 System Overview

- **AInteger.java** – Implements large integer arithmetic manually.
- **AFloat.java** – Implements high-precision floating-point arithmetic.
- **myInfArith.java** – Main CLI interface to perform operations.
- **build.xml** – Apache Ant script for compiling/running the project.
- **run.py** – Python script to run Java from Python.

# 2 Design Decisions

- **Manual Implementation of Arithmetic:** The project avoids Java's built-in `BigInteger` and `BigDecimal` to gain a deeper understanding of how arithmetic works internally. Strings are used to simulate digit-level operations.
- **Separation of Concerns:** Arithmetic for integers and floating points are handled in separate classes, `AInteger` and `AFloat`, to ensure cleaner logic and better maintainability.
- **Float Design Using Integer Backbone:** Floating-point numbers are split into integer and fractional parts and handled using `AInteger`, making reuse of logic possible and reducing duplication.
- **Command-Line Driven Execution:** The decision to build a CLI-based system using `myInfArith` ensures easy testing and integration with scripts like `run.py`.
- **Automation with Ant and Python:** Including `build.xml` and a Python wrapper was intended to streamline compilation and execution, especially for users not familiar with Java commands.

## Design Flow

The system flow can be summarized as:

```
run.py --> myInfArith --> (AInteger or AFloat)
```

# 3 Component Breakdown

## 3.1 AInteger.java

- Stores numbers as strings for unlimited size.
- Maintains a boolean for sign tracking.
- **addUnsigned()** and **subUnsigned()** do manual digit-wise operations.
- **mulUnsigned()** uses nested loops to simulate schoolbook multiplication.
- **divUnsigned()** implements basic long division with repeated subtraction.
- All the ***Unsigned** methods are private and as the name suggests these are defined for the unsigned integers. Each of these methods use logic which we first learnt in our school days.
- Sign-aware public methods: `add()`, `sub()`, `mul()`, `div()`.
- All the Sign-aware methods use Unsigned methods based on the requirement.

## 3.2 AFloat.java

- Splits float into integer and fractional parts using AInteger.
- Maintains a boolean for sign tracking.
- Decimal places tracked using `scale`.
- **Addition/Subtraction**: align scales, combine into whole numbers, operate, then re-separate.
- **Multiplication**: merge parts into full integers, multiply, then shift decimal.
- **Division**: scale numerator, perform integer division, extract decimal again.

### 3.3 myInfArith.java

- Accepts command-line arguments: `<type> <operation> <op1> <op2>`.
- Based on `type`, parses operands into AInteger or AFloat.
- Performs required operation and prints result.
- Handles cases of invalid inputs.

### 3.4 build.xml

- Ant script to automate build process.
- Targets: `clean`, `build`, `jar`, `run`.
- `build` compiles Java files to `build/classes`.

### 3.5 run.py

- Compiles Java files with `javac`.
- Parses and validates Python arguments.
- Runs Java main class using `subprocess.run()`.

## 4 Test Cases

### Test 1: Large Integer Addition

```
java myInfArith int add 999999999999999 1
```
Output: `1000000000000000`

### Test 2: Float Multiplication

```
java myInfArith float mul 1000000000.0 -2000000000000000000000000000000
```
Output: `-2000000000000000000000000000000000000000.0`

### Test 3: Integer Division by Zero

```
java myInfArith int div 5 0
```
Output: `Error:  Division by zero`

### Test 4: Float Subtraction

`java myInfArith float sub 2.50 1.25`
Output: `1.25`

### Test 5: Using Python Script

`python3 run.py int mul 10000 10000`
Output: `100000000`

### Test 6: float div having 30 digit precision

`python3 run.py float div 2 3`
Output: `0.666666666666666666666666666666`

## 5 Challenges and Solutions

- **Manual Carry/Borrow Handling**: Required careful iteration logic.
- **Zero Normalization**: Preventing `-0` and aligning scale in floats.
- **Division Precision**: Long division for floats needed artificial scale boosting to get the 30 decimal digit precision.

## 6 Pros and Cons

### Pros

- Handles arbitrarily large integers and high-precision decimals.
- Fully manual implementation without external libraries.
- Works consistently for both negative and positive values.
- Offers CLI and Python-based interfaces.

### Limitations

- Performance is not optimized for very large inputs.
- Division is slow due to repeated subtraction method.

- Error handling is basic and could be expanded.

# 7 Key Takeaways

- Arbitrary-precision logic teaches deep understanding of number operations.
- Handling signs and edge cases is as critical as core logic.
- Manual implementation improved my understanding of language internals and number systems.

# 8 Conclusion

Looking back at this project, it was more than just an implementation of arithmetic — it was an exploration of how numbers really work under the hood. By stripping away built-in conveniences and writing everything from scratch, I learned how to handle the tricky cases most libraries take care of for us. Whether it's carrying over digits in addition, normalizing decimals, or just ensuring '-0' doesn't slip through — **the devil is in the details**.

This system isn't perfect. It's not the fastest, nor the most feature-complete. But it's functional, educational, and open for extension. If I were to take this further, I'd add optimizations, better error handling, and maybe even support for things like exponentiation or modular arithmetic.

Overall, building this gave me a much deeper appreciation of the arithmetic engines we rely on every day — and confidence that, when needed, I can build my own.